Importación y exportación de ficheros en Java

TXT · CSV · JSON · XML

Acceso a Datos · Java · IntelliJ IDEA · Maven

Objetivo: entender qué ocurre realmente al convertir objetos Java en datos persistentes y recuperarlos después.

Docente: José Manuel Villar Ferradal

1. ¿Qué significan importar y exportar?

Exportar: transformar los datos que maneja el programa y escribirlos fuera de la memoria.

Importar: leer una representación externa y reconstruir datos u objetos que pueda usar Java.

El fichero es persistente: sigue existiendo cuando termina el programa.

Serializar = convertir un objeto/estructura en una representación almacenable o transmisible.

Deserializar = realizar el proceso inverso.

Objeto Java → representación → fichero

fichero → lectura → objeto Java

```
record Alumno(int id, String nombre) {}

Alumno a = new Alumno(7, "Lucía");

// El objeto existe ahora mismo en memoria.
// Para conservarlo habrá que representarlo
// en TXT, CSV, JSON, XML, binario...
```

2. La base: java.nio.file

Antes de hablar de formatos conviene dominar Path y Files.

`Path` representa una ruta; no contiene el contenido del fichero.

`Files` ofrece operaciones de lectura, escritura, copia, borrado y comprobación.

UTF-8 debe indicarse explícitamente cuando trabajamos con texto.

Para ficheros pequeños existen métodos muy compactos; para grandes, interesa procesar por flujo.

Ejemplo independiente

```
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;

Path ruta = Path.of("notas.txt");

Files.writeString(
    ruta,
    "Persistencia desde Java",
    StandardCharsets.UTF_8
);

String contenido =
    Files.readString(ruta, StandardCharsets.UTF_8);
```

3. TXT: el formato más directo

Ideal para enseñar flujos, codificación y delimitadores.

No existe un esquema estándar: nosotros decidimos cómo representar cada registro.

Puede usarse `Files.writeString`, `BufferedWriter`, `BufferedReader` o streams.

Problema: debemos programar nosotros el formato, el parseo, las validaciones y los casos especiales.

Exportación TXT

```
record Sensor(int id, String zona, double valor) {}

Sensor s = new Sensor(3, "Aula-2", 22.6);

try (BufferedWriter out = Files.newBufferedWriter(
        Path.of("sensor.txt"),
        StandardCharsets.UTF_8)) {

    out.write(s.id() + "|" + s.zona()
              + "|" + s.valor());
    out.newLine();
}
```

4. Importar TXT: leer, separar, convertir y validar

Leer una línea no basta: hay que interpretar su estructura.

`split()` separa campos cuando el formato es sencillo.

Las conversiones numéricas pueden lanzar `NumberFormatException`.

Una línea corrupta no debería provocar necesariamente la pérdida de toda la importación.

¿qué ocurre si el propio texto contiene el delimitador?

Importación TXT

```
try (BufferedReader in = Files.newBufferedReader(
        Path.of("sensor.txt"),
        StandardCharsets.UTF_8)) {

    String linea;
    while ((linea = in.readLine()) != null) {
        String[] p = linea.split("\\|", -1);

        if (p.length != 3) continue;

        int id = Integer.parseInt(p[0]);
        String zona = p[1];
        double valor = Double.parseDouble(p[2]);

        Sensor sensor = new Sensor(id, zona, valor);
    }
}
```

5. CSV: texto tabular con reglas

Muy apropiado para intercambio con hojas de cálculo y sistemas externos.

Cada línea suele representar un registro y cada columna un atributo.

Normalmente existe una fila de cabecera.

CSV parece sencillo, pero las comas, comillas y saltos de línea dentro de campos complican el parseo.

`String.split(",")` NO es un parser CSV completo.

Para ejemplos básicos puede hacerse manualmente; para producción conviene una librería.

Ejemplo de CSV

```
codigo,nombre,ciudad
A01,"Librería Norte",Ourense
A02,"Café, Plaza",Vigo
A03,"Academia ""Java""",Lugo

// La coma de "Café, Plaza" NO separa columnas.
// Las comillas dobles tienen reglas propias.
```

CSV es texto, pero no significa «hacer split por coma».

6. Librerías CSV en Java: ¿cuál usar?

Apache Commons CSV: API clara, madura y excelente para enseñanza y backend Java.

OpenCSV: muy cómoda para mapear filas a beans y para proyectos sencillos.

Jackson Dataformat CSV: interesante si el proyecto ya usa el ecosistema Jackson.

Para aplicaciones reales: evita implementar a mano todas las reglas de escaping salvo que el formato sea controlado y trivial.

Dependencia Maven

```
<dependency>
  <groupId>org.apache.commons</groupId>
  <artifactId>commons-csv</artifactId>
  <version>...</version>
</dependency>

// Maven descargará la biblioteca
// y sus dependencias necesarias.
```

Recomendación didáctica: Apache Commons CSV

7. JSON: datos estructurados

Natural para APIs, configuración e intercambio entre aplicaciones.

Representa objetos, arrays, números, cadenas, booleanos y null.

Se adapta mucho mejor que TXT/CSV a estructuras anidadas.

Es legible por personas y ampliamente interoperable.

En Java normalmente no escribimos JSON concatenando cadenas: usamos una librería de serialización.

Ejemplo JSON

```
{
  "codigo": "A-17",
  "titulo": "Programación",
  "activo": true,
  "etiquetas": [
    "java",
    "ficheros",
    "datos"
  ]
}
```

Objeto Java ⇄ JSON

8. Librerías JSON: Jackson, Gson y JSON-B

Jackson: opción muy completa; excelente para backend Java y estructuras complejas.

Gson: API muy sencilla y fácil de introducir a principiantes.

JSON-B (Jakarta JSON Binding): estándar del ecosistema Jakarta; útil para enseñar APIs estandarizadas.

Moshi: alternativa moderna, especialmente conocida en ecosistemas Android/Kotlin.

Jackson es una elección sólida; Gson es una alternativa si se prioriza simplicidad inicial.

Jackson: ida y vuelta

```
record Libro(String isbn, String titulo) {}

ObjectMapper mapper = new ObjectMapper();

Libro libro =
    new Libro("978-01", "Java práctico");

// Java → JSON
mapper.writeValue(
    Path.of("libro.json").toFile(),
    libro
);

// JSON → Java
Libro copia = mapper.readValue(
    Path.of("libro.json").toFile(),
    Libro.class
);
```

9. ¿Qué es Maven y qué es pom.xml?

Maven es una herramienta de construcción y gestión de proyectos Java.

`pom.xml` significa Project Object Model.

Describe el proyecto: identificador, versión, Java utilizado, dependencias, plugins y proceso de construcción.

El POM permite que otro desarrollador abra el proyecto y Maven obtenga automáticamente las bibliotecas declaradas.

TXT puro no necesita dependencia externa; CSV/JSON pueden beneficiarse de librerías añadidas al POM.

Estructura conceptual del POM

```
<project>
  ...
  <properties>
    <maven.compiler.release>21</maven.compiler.release>
  </properties>

  <dependencies>
    <dependency>
      <groupId>...</groupId>
      <artifactId>...</artifactId>
      <version>...</version>
    </dependency>
  </dependencies>
</project>
```

10. ¿Qué ocurre al añadir una dependencia?

IntelliJ detecta los cambios del `pom.xml` y Maven resuelve la dependencia.

Maven busca el artefacto en los repositorios configurados, normalmente Maven Central.

El JAR pasa a formar parte del classpath del proyecto.

Entonces podemos importar sus clases desde Java.

No debemos copiar manualmente JARs al proyecto si Maven ya gestiona las dependencias.

```
// Después de declarar Jackson en Maven:
import com.fasterxml.jackson.databind.ObjectMapper;

ObjectMapper mapper = new ObjectMapper();

// Si la dependencia no está disponible,
// IntelliJ no podrá resolver ObjectMapper.
```

pom.xml → Maven → JAR → classpath → import en Java

12. Excepciones y robustez

Importar datos externos significa asumir que pueden venir mal.

`IOException`: problemas al abrir, leer o escribir.

`NoSuchFileException`: el fichero no existe.

`NumberFormatException`: un campo esperado como número contiene otro valor.

Los parsers JSON/CSV pueden lanzar sus propias excepciones de formato.

Conviene validar antes de sustituir los datos actuales por los importados.

En una interfaz gráfica, mostrar mensajes comprensibles al usuario; registrar el detalle técnico aparte.

```
try {
    List<String> lineas =
        Files.readAllLines(Path.of("entrada.txt"));

    // Procesar y validar...

} catch (NoSuchFileException e) {
    System.err.println("No existe el fichero");
} catch (IOException e) {
    System.err.println(
        "No se pudo leer: " + e.getMessage()
    );
}
```

13. Buenas prácticas que merece la pena enseñar

No confiar en el contenido de un fichero externo.

Definir y documentar claramente el formato esperado.

Usar UTF-8 de forma consistente.

No cargar ficheros gigantes completamente en memoria si pueden procesarse línea a línea.

Evitar sobrescribir un fichero importante sin confirmación o copia de seguridad.

Separar la lógica de interfaz, dominio y persistencia.

Probar casos límite: fichero vacío, línea corrupta, duplicados, campos ausentes, comillas, caracteres Unicode y números inválidos.



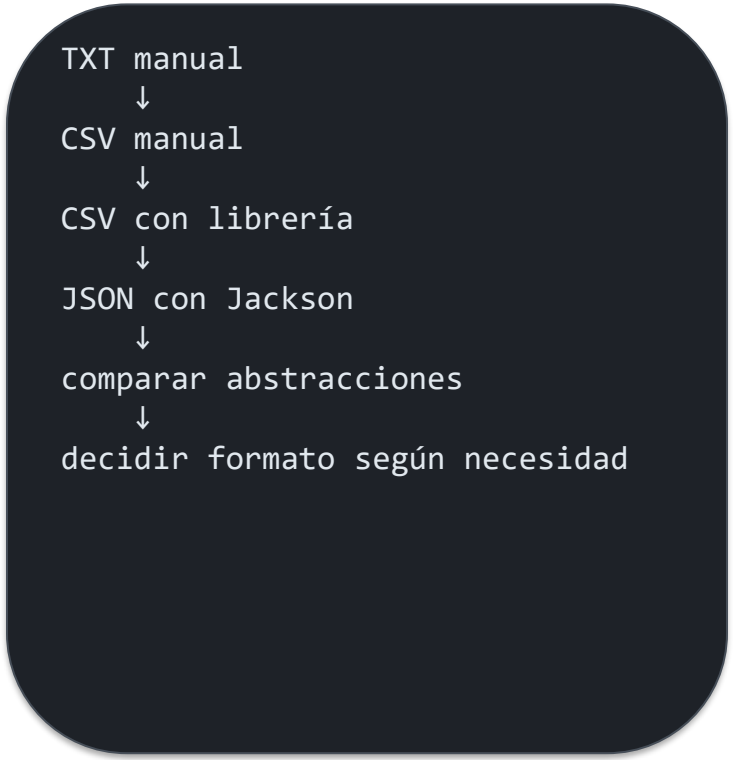
Interfaz → lógica → fichero

Cada capa tiene una responsabilidad.

14. Secuencia didáctica recomendada

1. Escribir y leer una cadena con `Files`.
2. Exportar varios objetos manualmente a TXT.
3. Importar TXT y trabajar validaciones/excepciones.
4. Introducir CSV y descubrir por qué `split()` no resuelve todos los casos.
5. Incorporar Apache Commons CSV y comparar código manual vs librería.
6. Introducir JSON y serialización/deserialización con Jackson.
7. Añadir Maven/pom.xml y explicar gestión de dependencias.
8. Integrar selector de formato en una GUI y probar ficheros correctos e incorrectos.

Evolución



Ideas clave

TXT enseña el mecanismo.

CSV enseña que los formatos tienen reglas.

JSON enseña serialización estructurada.

Maven enseña a gestionar dependencias de forma reproducible.

Las librerías reducen código, pero conviene entender primero el problema que resuelven.

XML: datos estructurados y autodescriptivos

Otro formato de intercambio muy habitual en sistemas empresariales, configuración e integraciones.

XML organiza la información mediante etiquetas y una estructura jerárquica.

Puede representar atributos, elementos anidados y colecciones.

Es más verboso que JSON, pero dispone de tecnologías maduras de validación y transformación.

En Java existen APIs estándar como DOM, SAX y StAX, además de binding de objetos mediante JAXB.

Si el proyecto ya utiliza Jackson, `jackson-dataformat-xml` permite reutilizar un modelo de programación parecido al de JSON.

Ejemplo XML

```xml
<curso codigo="AD01">
  <titulo>Acceso a Datos</titulo>
  <horas>120</horas>
  <profesor>
    <nombre>José Manuel</nombre>
  </profesor>
</curso>
```

texto estructurado: las etiquetas describen el significado de los

XML en Java: APIs y configuración Maven

DOM: carga el documento completo como árbol. Cómodo para documentos pequeños y modificaciones.

SAX: procesamiento dirigido por eventos; consume poca memoria, pero es menos intuitivo.

StAX: lectura/escritura por flujo con control desde el código; útil para XML grandes.

JAXB: mapea clases Java ↔ XML mediante anotaciones; en Java moderno suele añadirse como dependencia.

Jackson XML: opción práctica si ya usamos Jackson para JSON; se añade `jackson-dataformat-xml` al `pom.xml`.

Jackson XML

```
<dependency>
  <groupId>com.fasterxml.jackson.dataformat</groupId>
  <artifactId>jackson-dataformat-xml</artifactId>
  <version>...</version>
</dependency>

// Java
XmlMapper xml = new XmlMapper();

Informe informe = new Informe("Septiembre", 24);

// Java → XML
xml.writeValue(new File("informe.xml"), informe);

// XML → Java
Informe copia = xml.readValue(
    new File("informe.xml"),
    Informe.class
);
```

Comparativa final: TXT, CSV, JSON y XML

Formato	Estructura	Librería recomendada	Fortaleza
TXT	Libre / propia	JDK (`Files`, buffers)	Aprender E/S
CSV	Tabular	Apache Commons CSV	Intercambio tabular
JSON	Jerárquica	Jackson	APIs y objetos
XML	Jerárquica + etiquetas	Jackson XML / JAXB	Integración y validación

Elección recomendada: usar el formato que mejor represente los datos y el sistema con el que debemos interoperar.